

Documentation report for Operating Systems Assignment 2

ATHANASIOS GOURDOMICHALIS

The general structure across all three mysh implementations consists of the main() function containing an infinite while(1) loop that continuously displays the prompt "\$ " for the ongoing execution of the shell. This loop terminates upon receiving the "exit" command or encountering an unrecoverable error (via break), such as an execution failure in exec(), etc. Furthermore, in the case of invalid input, the shells do not hang or crash, nor do they print anything to stdout; instead, error messages are routed to stderr using perror() (e.g., "Command not found"). I have implemented all parts of the assignment, and regarding the automated test scripts for submission, everything passes correctly. Finally, comprehensive inline comments have been provided throughout most of the codebase for further clarification.

Issues Encountered During Implementation & Solutions. For mysh[123].c:

1. **mysh1.c:** I read the program name from standard input using getline(3), passing a char *input buffer pointer, a size_t length for the buffer capacity, and storing the return value in an ssize_t line variable (which includes the trailing newline \n).
 - **Issue & Fix:** Initially, I faced failures on automated Tests 15–18 because I had not accounted for input with leading whitespaces (e.g., " ____ls" instead of "ls"). **I resolved this by using strtok(3)-as suggested in the project specifications-passing input and " \t\n" as the delimiter set (spaces, tabs, and newlines). After testing various edge cases before and after the fix, the issue was resolved and the tests passed.**
 - **Empty Commands & fork() Failures:** For empty lines or rare fork() errors, the execution simply proceeds to the next iteration using continue.
 - **Process Execution:** Inside the child process, I chose execlp(3) because we only pass a single parameter list (the command name followed by NULL), and it automatically searches the directories listed in the \$PATH environment variable.
 - **Synchronization:** The parent process calls waitpid(3) to wait for the specific child process to finish before displaying the prompt again for the next command.
 - **Memory Management:** Finally, I call free(input) at the end. **Researching C library behavior revealed that getline(3) internally allocates dynamic memory via malloc when passed a NULL buffer or zero size, so freeing it prevents memory leaks.**
2. **mysh2.c:** Building upon mysh1, mysh2 adds support for command-line arguments.
 - **Argument Parsing:** Input tokens are initially extracted using strtok(3) with " \t\n" as delimiters. Inside a while loop, subsequent tokens are parsed and sequentially stored into a dynamically allocated arguments array.

- **Process Execution:** Because command-line arguments are stored as a dynamically populated array of pointers, I replaced `execlp(3)` with `execvp(3)`.
- **Built-in cd Command:** I added explicit logic to handle the `cd` built-in command:
 - Error checking for non-existent files or directories when `chdir(arguments[1]) != 0`.
 - Handling cases where the user supplies `cd` without an argument (`arguments[1] == NULL`).
 - Optionally inspecting the current working directory after a successful `chdir` call using `getcwd(3)` with a `char cwd[1024]` buffer.
- **Memory Management:** The process creation logic using `fork()` and checking the return `pid_t` remains identical to `mysh1`. However, whenever the control flow changes (e.g., exiting, handling `cd`, or encountering errors during `fork/exec`), I explicitly call `free()` on the arguments array to avoid memory leaks before entering the next loop iteration.
- **Issue & Fix:** The primary issue I encountered was initially using a fixed-size static array for the `exec` argument vector (e.g., 64 or 80 elements). This resulted in test failures until I received the automated test hints:

***"Test 10/37: fail... (HINT: Are you using dynamic allocation?)" and
"Test 21/37: fail... (HINT: Are you using dynamic allocation?)"***

Switching to dynamic memory allocation resolved these issues. Furthermore, studying the manual pages for `chdir(2)` and `getcwd(3)` proved invaluable for implementing directory navigation correctly.

In particular, the following selected lines proved especially helpful:

NAME `chdir` - change working directory

LIBRARY Standard C library (`libc`, `-lc`)

SYNOPSIS `#include <unistd.h>`
`int chdir(const char *path);`

DESCRIPTION `chdir()` changes the current working directory of the calling process to the directory specified in `path`. **RETURN VALUE** On success, zero is returned. On error, `-1` is returned, and `errno` is set to indicate the error.

NOTES The current working directory is the starting point for interpreting relative pathnames (those not starting with `/`). A child process created via `fork(2)` inherits its parent's current working directory. The current working directory is left unchanged by `execve(2)`.

NAME `getcwd` - get current working directory **LIBRARY** Standard C library (`libc`, `-lc`)

SYNOPSIS `#include <unistd.h>`
`char *getcwd(char buf[size], rsize_t size);`

DESCRIPTION These functions return a null-terminated string containing an absolute pathname that is the current working directory of the calling process. The pathname is returned as the function result and via the argument `buf`, if present. The `getcwd()` function copies an absolute pathname of the current working directory to the array pointed to by `buf`, which is of length `size`. If the length of the absolute pathname of the current working directory, including the terminating null byte, exceeds `size` bytes, `NULL` is returned, and `errno` is set to `ERANGE`; an application should check for this error, and allocate a larger buffer if necessary. As an extension to the POSIX.1-2001 standard, `glibc's getcwd()` allocates the buffer dynamically using `malloc(3)` if

buf is NULL. In this case, the allocated buffer has the length size unless size is zero, when buf is allocated as big as necessary. The caller should free(3) the returned buffer.

RETURN VALUE On success, these functions return a pointer to a string containing the pathname of the current working directory. In the case of getcwd() and getwd() this is the same value as buf. On failure, these functions return NULL, and errno is set to indicate the error. The contents of the array pointed to by buf are undefined on error.

ATTRIBUTES For an explanation of the terms used in this section, see attributes(7).

Interface | Attribute | Value |

getcwd(), getwd() | Thread safety | MT-Safe |

get_current_dir_name() | Thread safety | MT-Safe env |

NOTES These functions are often used to save the location of the current working directory for the purpose of returning to it later. Opening the current directory (".") and calling fchdir(2) to return is usually a faster and more reliable alternative when sufficiently many file descriptors are available, especially on platforms other than Linux.

3. **mysh.3:** Building upon mysh1 and mysh2, mysh3 adds support for **pipes**.

- **Parsing & Splitting:** After reading user input via getline(3), I iterate through the char array (C-string) searching for the pipe delimiter character '|'. If no pipe is found, execution falls back seamlessly to the exact same logic used in mysh2.c. If a pipe is found, I replace the '|' character with a null terminator ('\0'). This allows me to assign char *cmd_1 to the first command preceding the pipe (as reading stops at '\0'), while char *cmd_2 points to the second command immediately following the '|', leveraging the contiguous memory layout of the original input buffer. I then apply the exact parsing logic from mysh2 using strtok(3) on both individual commands, including identical error checks and dynamic memory allocations/deallocations.
- **Process Creation & Inter-Process Communication (IPC):** To execute the piped commands, I declare two process IDs (pid_t pid1, pid2) for the two child processes and an integer array int fd[2] to store the read and write file descriptors for the pipe. Appropriate error handling is included for pipe(2) and both fork(2) calls, along with proper dynamic memory cleanups.
- **Redirection using dup2(2):** The primary challenge (though no automated test cases failed) was understanding how to connect the two processes via dup2(2) so that the stdout of the first command flows into the stdin of the second command:
 - In the **first child process** (executing cmd_1), I redirect standard output (stdout) to fd[1] (the write end of the pipe) using dup2(2).
 - In the **second child process** (executing cmd_2), I redirect standard input (stdin) to fd[0] (the read end of the pipe) using dup2(2).
- **Closing File Descriptors:** Immediately after invoking dup2(2), unused file descriptors are closed within each child. In the first child, fd[0] is closed because it does not read from the pipe, and fd[1] is closed as it is redundant after duplication. In the second child, fd[1] is closed because it does not write to the pipe, and fd[0] is closed after dup2(2).
- **Parent Process Cleanup:** Most importantly, both pipe file descriptors (fd[0] and fd[1]) **must also be closed in the parent process** outside the child conditional blocks. If the parent keeps fd[1] open, the second child process will hang

indefinitely waiting for more input (EOF will never be sent), assuming the parent might still write to the pipe. This causes the shell to freeze/hang and fail to print the next prompt (" \$ ").

In particular, the following selected lines from the manual pages for dup(2) and dup2(2) proved especially helpful:

dup(2) System Calls Manual dup(2)

NAME dup, dup2 - duplicate a file descriptor

DESCRIPTION The dup() system call allocates a new file descriptor that refers to the same open file description as the descriptor oldfd. (For an explanation of open file descriptions, see open(2).) The new file descriptor number is guaranteed to be the lowest-numbered file descriptor that was unused in the calling process.

After a successful return, the old and new file descriptors may be used interchangeably. Since the two file descriptors refer to the same open file description, they share file offset and file status flags; for example, if the file offset is modified by using lseek(2) on one of the file descriptors, the offset is also changed for the other file descriptor. The two file descriptors do not share file descriptor flags (the close-on-exec flag). The close-on-exec flag (FD_CLOEXEC; see fcntl(2)) for the duplicate descriptor is off.

dup2() The dup2() system call performs the same task as dup(), but instead of using the lowest-numbered unused file descriptor, it uses the file descriptor number specified in newfd. In other words, the file descriptor newfd is adjusted so that it now refers to the same open file description as oldfd. **DESCRIPTION** The dup() system call allocates a new file descriptor that refers to the same open file description as the descriptor oldfd. (For an explanation of open file descriptions, see open(2).) The new file descriptor number is guaranteed to be the lowest-numbered file descriptor that was unused in the calling process.

After a successful return, the old and new file descriptors may be used interchangeably. Since the two file descriptors refer to the same open file description, they share file offset and file status flags; for example, if the file offset is modified by using lseek(2) on one of the file descriptors, the offset is also changed for the other file descriptor. The two file descriptors do not share file descriptor flags (the close-on-exec flag). The close-on-exec flag (FD_CLOEXEC; see fcntl(2)) for the duplicate descriptor is off.

dup2() The dup2() system call performs the same task as dup(), but instead of using the lowest-numbered unused file descriptor, it uses the file descriptor number specified in newfd. In other words, the file descriptor newfd is adjusted so that it now refers to the same open file description as oldfd. **RETURN VALUE** On success, these system calls return the new file descriptor. On error, -1 is returned, and errno is set to indicate the error.